

AHDS - STM32L0 MODULAR DEVELOPMENT & PROTOTYPING PLATFORM

Full Embedded Hardware & Software Project Documentation

Main MCU Board + DB37 Connection Board + STM32 Firmware + C# PC Configuration Tool

A reusable STM32L0 development platform combining custom hardware, embedded C firmware, a UART command protocol, persistent configuration, and a Windows PC configuration/diagnostic application.

Primary MCU	STM32L062K8T6 (LQFP32)
Platform concept	Reusable STM32L0 hardware, firmware and PC-software development system
Hardware status	Schematics and PCBs completed; boards assembled and used
Expansion	37-pin board-to-board connection to a dedicated breakout / connection PCB
Main interfaces	SWD, USB, UART, SPI, I2C, GPIO, ADC, DAC, EXTI
Project scope	Hardware documentation, STM32 firmware, UART protocol and Windows PC terminal

Project positioning

A full embedded development project combining a reusable STM32L0 hardware platform, DB37 expansion, public STM32 firmware, UART configuration protocol, Windows PC software and technical documentation.

1. Project Purpose

The platform was created to avoid rebuilding the same MCU support circuitry for every prototype. Instead, the main board provides a reusable STM32L0 core with common interfaces and analog resources already available, while a separate DB37 connection board makes the signals easy to access during laboratory work and firmware development.

The assembled implementation uses the STM32L062K8T6. The design concept can also be used with other STM32L0 devices that are pin-compatible with the selected LQFP32 footprint. Exact device substitution should be checked against the specific MCU datasheet and alternate-function mapping before use.

System Concept

MAIN MCU BOARD	DB37 INTERCONNECT	CONNECTION BOARD
STM32L0 core Power regulation Analog conditioning USB / SWD / UART / SPI / I2C Jumpers and local peripherals	37-pin link Carries MCU and peripheral signals Keeps the main board reusable	Breaks signals into accessible groups GPIO / ADC / DAC UART / SPI / I2C SWD / USB Power rails and test connections

2. Main Hardware Functions

MCU core: STM32L062K8T6 with GPIO and alternate-function access routed for prototyping.

Programming and debug: Dedicated ST-LINK/SWD connection using SWCLK, SWDIO, 3.3 V and GND.

Digital communication: USART, SPI and I2C signals available for external modules and firmware experiments.

USB: USB D+ and D- lines are available on the platform.

Analog section: Multiple ADC paths, DAC access and operational-amplifier stages based on TLV2374/TLV2372 devices.

I/O expansion: PCA9654 I2C GPIO expander provides additional digital I/O resources.

Sensors and modules: Connections include a motion sensor input, IR receiver and a JDY communication-module interface.

Clocking and system support: 32.768 kHz crystal provision, reset circuitry, BOOT0 control and configurable jumpers.

Power: 12 V input with on-board 3.3 V regulation and local decoupling for the MCU and peripherals.

3. Prototyping and Configuration Strategy

The platform is designed around configurable routing rather than a single fixed application. Jumpers and 0-ohm links allow selected MCU pins to be connected to analog channels, sensor functions, or external connectors as required. This makes the same assembled board useful across different firmware and hardware experiments.

Area	Examples	Purpose
Analog routing	PA0, PA1, PA4, PA5, PA6 / ADC and DAC paths	Select direct GPIO or analog-function routing.
Sensor routing	PA7 / motion input	Reuse the same MCU pin for different test configurations.
I2C	SCL1 / SDA1 and optional pull-up connection	Support external I2C devices while avoiding duplicate pull-ups when necessary.
Expansion I/O	P4-P7 from the I2C expander	Add accessible digital I/O without consuming additional MCU pins.
Board-to-board access	DB37 plus grouped external connectors	Bring MCU and peripheral signals to the connection board for practical laboratory access.

4. Embedded Firmware and Real-Project Use

The hardware platform has already been used in a real engineering application, demonstrating that the design moved beyond schematic and PCB development into practical use. The original employer-related firmware is proprietary and is intentionally excluded from this public project.

The public version instead includes independently developed firmware for the same STM32L0 platform. This firmware provides a practical motion-control example using the STM32L062, with a motion-sensor input, DAC output control, RTC-based timing, internal Data EEPROM, status/heartbeat functions, and a UART command interface for configuration and diagnostics.

Public repository note

Only hardware files and source code that are independently owned and safe to publish are included. The original employer-related application firmware remains excluded; the GitHub repository uses the independent STM32L0 motion-control firmware and the independently developed PC terminal instead.

5. Firmware Architecture and UART Configuration Protocol

The public firmware is written in C for STM32L0 using the STM32 HAL. It is designed around non-blocking peripheral services and runtime configuration rather than a single hard-coded operating point. The motion input is handled through EXTI, brightness is controlled through the DAC, hold timing is maintained with the RTC WakeUp timer, and selected operating parameters are stored in the STM32L0 internal Data EEPROM.

Firmware area	Implementation	Project role
---------------	----------------	--------------

Email: ahds@engineer.com

Firmware area	Implementation	Project role
Motion input	PB1 / EXTI with debounce and refractory timing	Detect motion while reducing false or repeated triggers.
Output control	12-bit DAC with logical High / Low scaling	Apply programmable output levels for the motion-control example.
Timing	RTC WakeUp timer at 1 Hz	Maintain a configurable hold interval without blocking the main application.
Persistent settings	STM32L0 Data EEPROM	Store device ID, hold duration, Low level, High reference and motion mode.
UART protocol	<,CMD,ARG,...,> ASCII command frames	Read, modify and verify operating parameters from a PC terminal.
Diagnostics	INFO, HELP, status, heartbeat and test commands	Support bring-up, service and functional testing.

Example command frames

```

<, INFO, >
<, SETDUR,MM,> / <, GETDUR, >
<, SETHI,PP,> / <, GETHI, >
<, SETDIM,PP,> / <, GETDIM, >
<, SETID,NNNNNNNN,> / <, GETID, >
<, MOTION,MODE,INV|NORM[, SAVE], >
<, DAC,PP,> / <, DACTEST,TT,>

```

6. Windows PC Configuration and Diagnostic Tool

A dedicated Windows application was developed in C# with Visual Studio / Windows Forms to communicate with the STM32 board through a serial COM port. The application is not a firmware flasher; it is a configuration, test and diagnostic tool that exchanges commands with the running STM32 firmware and can modify parameters that the MCU stores in Data EEPROM.

PC terminal function	Implementation / purpose
Serial setup	COM port, baud rate, parity, stop bits and data-bit selection.
Terminal modes	Text / Hex transmission, optional CR / LF, manual Send, console display and zoom.
Predefined command groups	General commands, motion programming, motion tests, DAC tests, heartbeat/SystemTick tests and logic-mode changes.
Board configuration panel	Read / Write controls for device ID, High level, hold time and Low level.
Response parsing	The application recognizes MCU responses such as DUR, DIM, HI and programmed ID and updates the GUI automatically.
Verification	After writing settings, the application can issue GET commands and wait asynchronously for returned values.
Diagnostics	INFO-based communication check to confirm that the selected COM port can reach the MCU.
Logs	Console history can be cleared or saved to a timestamped text file.

System-level architecture

Sensor / I/O	STM32L0 Hardware	Embedded Firmware	Serial Link	PC Application
Motion sensor and external peripherals	Custom main board + DB37 expansion	EXTI / DAC / RTC / EEPROM / UART	ASCII command protocol over COM / UART	C# configuration, test and diagnostics

Together, the custom hardware, STM32 firmware and Windows application form a complete development chain: physical I/O and sensors are handled by the MCU, operating parameters are retained in EEPROM, and the PC software provides a practical service interface for configuration and testing.



7. GitHub Repository and Source-Code Availability

The GitHub repository for this project is intended to contain the complete public project package: hardware documentation, the independently developed STM32L0 firmware, and the Visual Studio C# terminal source code. This allows the platform to be reviewed as a complete embedded system rather than as a schematic-only project.

- Main STM32L0 board schematic and hardware documentation.
- DB37 connection / breakout board schematic, pinout and jumper documentation.
- Independent STM32L0 firmware source files for the motion-control example.
- Visual Studio / C# PC configuration and diagnostic terminal source files.
- UART command protocol examples and project documentation.
- Future examples can be added without changing the core platform hardware.

Email: ahds@engineer.com

8. Assembled Hardware Platform

The photographs below show the manufactured and assembled prototype hardware. The main STM32L0 controller board and the dedicated DB37 connection / breakout board are shown both as separate modules and as a complete connected system.



Main controller board and DB37 connection / breakout board as separate assembled modules.



Complete platform connected through the DB37 interface for direct access to MCU and peripheral signals.

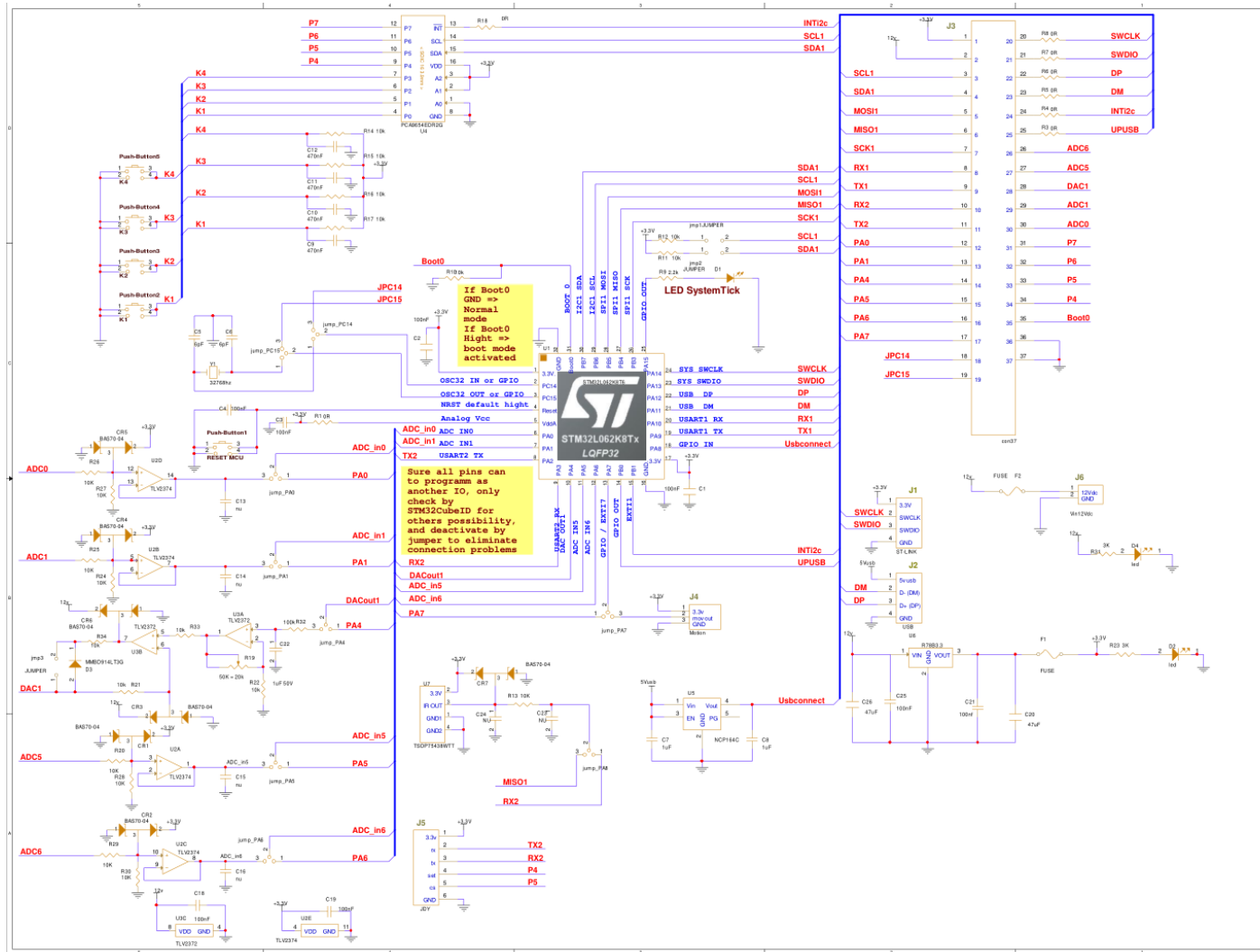
The assembled hardware confirms that the project progressed from schematic and PCB design to a physical, reusable development platform.

Email: ahds@engineer.com

Appendix A - Main STM32L0 Board Schematic

Main controller board centered on STM32L062K8T6 with power, analog, communication, programming and expansion circuitry.

Email: ahds@engineer.com



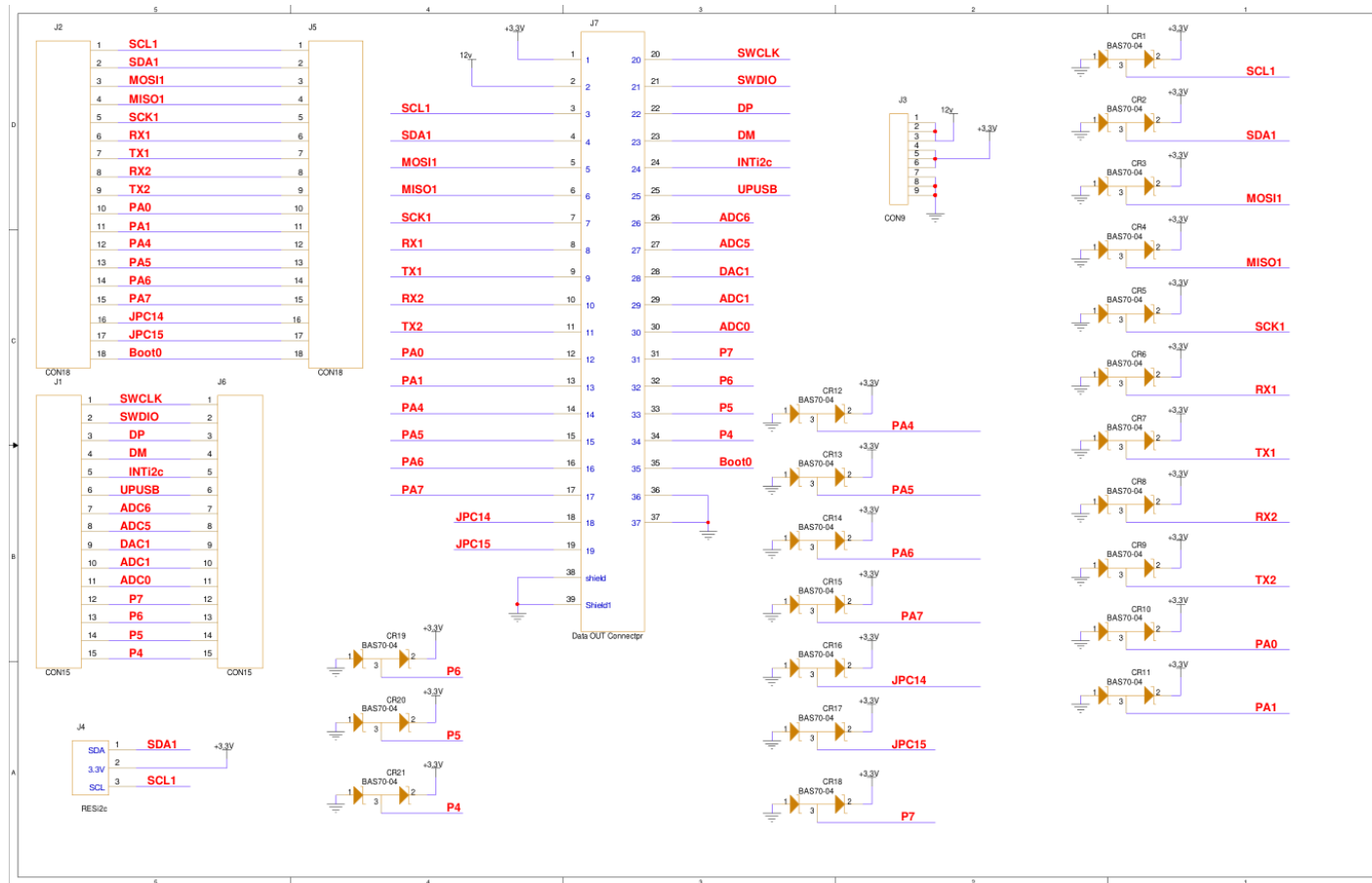
Main-board schematic. The large 37-pin connector provides the link to the separate connection / breakout board.

Email: ahds@engineer.com

Appendix B - Connection Board Schematic

DB37 signal distribution, grouped external connectors, protection components and optional I2C pull-up access.

Email: ahds@engineer.com



Connection-board schematic - signal distribution from the DB37 interface to accessible external connectors.

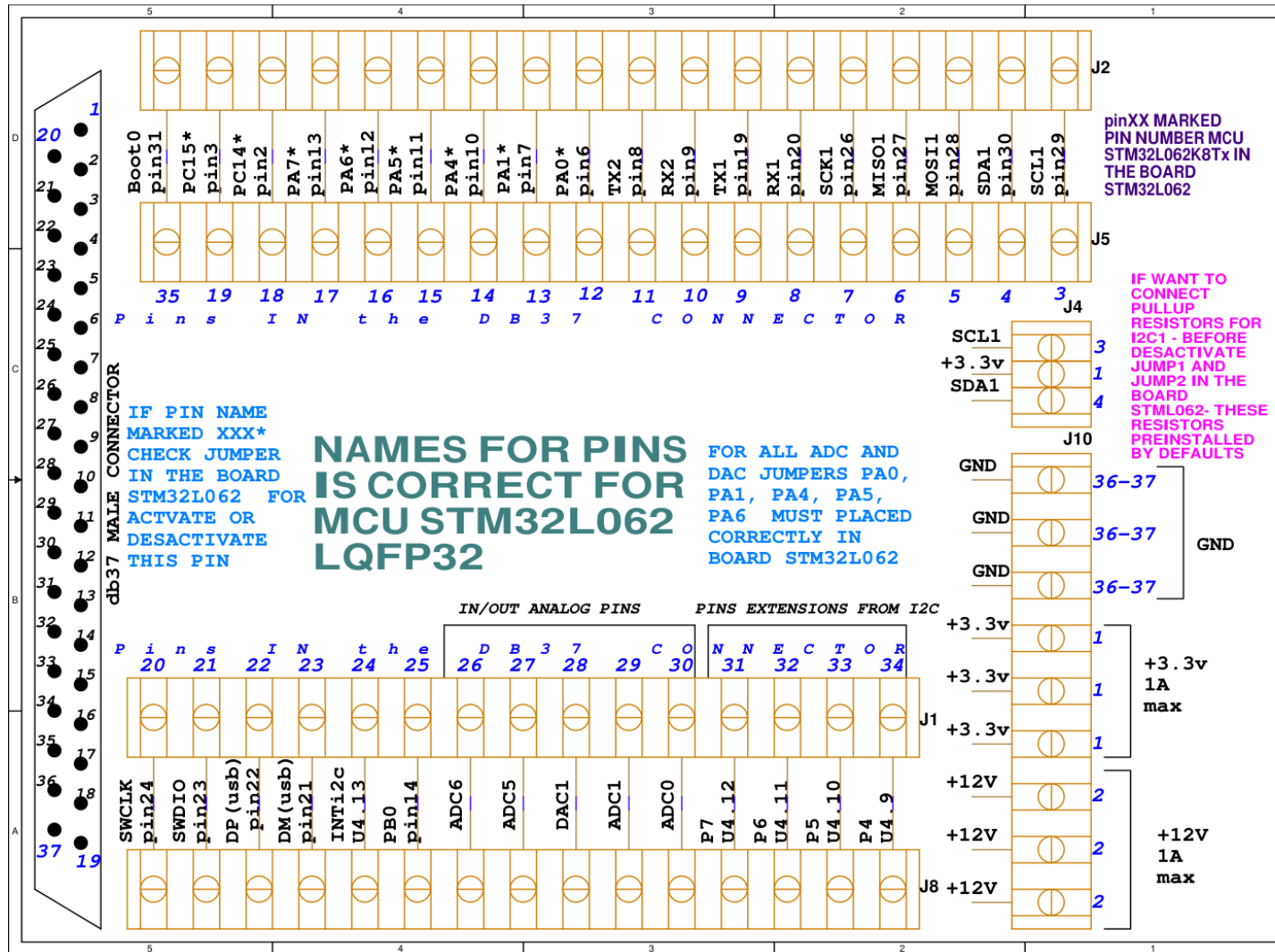
Email: ahds@engineer.com

Appendix C - Connection Board Pinout

Pin mapping for STM32L062 signals, analog functions, communication interfaces, power rails and I2C-expander outputs.



Email: ahds@engineer.com



Connection-board pinout and connector mapping.

Email: ahds@engineer.com

Appendix D - Jumper and Button Configuration

Configuration reference for jumpers, analog routing, motion input and local push buttons.

Email: ahds@engineer.com

	5	4	3	2	1
1					
2	SWITCH	PIN 1	PIN 2	PIN3	INFO
3	JUMP1	X	X	N/A	on/off RESISTOR 10K SCL1
4					
5	JUMP2	X	X	N/A	on/off RESISTOR 10K SDA1
6					
7	JUMP3	X	X	N/A	on/off DIODE DAC1
8					
9	JUMP PA0	1-2 PA0 ready	2-2 off ALL	3-2 ADC0 ready	CONNECT PA0 or ADC0 TO EXT
10					
11	JUMP PA1	1-2 PA1 ready	2-2 off ALL	3-2 ADC1 ready	CONNECT PA1 or ADC1 TO EXT
12					
13	JUMP PA4	1-2 PA4 ready	2-2 off ALL	3-2 DAC1 ready	CONNECT PA4 or DAC1 TO EXT
14					
15	JUMP PA5	1-2 PA5 ready	2-2 off ALL	3-2 ADC5 ready	CONNECT PA5 or ADC5 TO EXT
16					
17	JUMP PA6	1-2 PA6 ready	2-2 off ALL	3-2 ADC6 ready	CONNECT PA6 or ADC6 TO EXT
18					
19	JUMP PA7	1-2 PA7 ready	2-2 off ALL	3-2 MOTION ready	CONNECT PA7 or MOTION READY
20					
21	JUMP PA8	1-2 RX2(PA3) ready IR	2-2 off ALL	3-2 MISO1(PB4) ready IR	CONNECT PA7 or MOTION READY
22					
23	Push Button1				reset mcu restart
24					
25	K1				press P0=0 U4(i2c) default = 1
26					
27	K2				press P1=0 U4(i2c) default = 1
28					
29	K3				press P2=0 U4(i2c) default = 1
30					
31	K4				press P3=0 U4(i2c) default = 1
32					

CLICK BUTTON

Jumper / switch configuration table used when adapting the platform to different prototype setups.

9. Project Status and Next Steps

The hardware platform is already designed, manufactured and assembled. The public project package now combines the hardware documentation, photographs of the assembled two-board system, an independently developed STM32L0 motion-control firmware example, and a C# Windows configuration / diagnostic application. The original employer-related firmware remains excluded from the public repository.

- Add additional close-up, test-bench and application photographs as the project evolves.
- Add a concise bring-up guide covering power, SWD, firmware load, UART connection and first configuration test.
- Maintain the independent STM32L062 firmware example and document future revisions.
- Add further application examples, such as ADC acquisition, I2C peripherals, SPI devices or additional sensor tests.
- Keep the main platform reusable so that future firmware and PC-tool examples can be added without redesigning the core hardware.

Project classification: Full Embedded Hardware Project - STM32L0 Modular Development & Prototyping Platform with Firmware and PC Configuration Tool

Prepared as project documentation for GitHub, LinkedIn technical portfolio use, and future STM32L0 development examples.